Desarrollo de API REST con Laravel

Guía Académica - Programación Avanzada | Eloquent ORM & Postman

Estimados estudiantes, en esta sesión aprenderemos a construir y configurar una API RESTful robusta utilizando Laravel. Nos enfocaremos en las buenas prácticas, la correcta configuración del entorno y el uso del ORM Eloquent para interactuar con la base de datos de manera orientada a objetos.

1. Configuración Inicial del Proyecto y API

En las versiones modernas de Laravel, las rutas de API no vienen habilitadas por defecto para mantener el framework ligero. El primer paso es crear el proyecto y publicar los archivos necesarios para la API.

```
# 1. Crear el proyecto Laravel
composer create-project laravel/laravel api-personas

# 2. Entrar al directorio
cd api-personas

# 3. Instalar el andamiaje para la API (Genera routes/api.php)
php artisan install:api
```

Configuración de la Base de Datos

Abre el archivo `.env` en la raíz de tu proyecto y configura las credenciales de tu base de datos. Para este laboratorio, usaremos MySQL:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=laravel_api_db
DB_USERNAME=root
DB_PASSWORD=tu_contraseña
```

No olvides crear la base de datos `laravel_api_db` en tu gestor de base de datos (como phpMyAdmin o DBeaver) antes de continuar.

2. Implementación de Migraciones y Modelo Eloquent

Para construir nuestro recurso **Persona**, usaremos el generador de código de Laravel (Artisan) para crear el Modelo, la Migración y el Controlador en un solo comando.

```
php artisan make:model Persona -mc
```

El flag `-mc` indica que queremos crear una **M**igración y un **C**ontrolador asociados al modelo.

Definiendo la Migración

Abre el archivo generado en `database/migrations/YYYY_MM_DD_XXXXXX_create_personas_table.php`. Aquí definiremos la estructura de nuestra tabla usando programación fluida.

```
public function up(): void
{
    Schema::create('personas', function (Blueprint $table) {
        $table->id();
        $table->string('nombres', 100);
        $table->string('apellidos', 100);
        $table->string('email')->unique();
        $table->string('telefono', 20)->nullable();
        $table->timestamps(); // Crea created_at y updated_at
    });
}
```

Ejecuta la migración para crear la tabla en la base de datos:

```
php artisan migrate
```

Configurando el Modelo (Asignación Masiva)

Abre `app/Models/Persona.php`. Para que Eloquent nos permita insertar datos de forma segura desde nuestra API, debemos definir la propiedad `$fillable`. Si omitimos este paso, Laravel bloqueará la inserción por seguridad (protección contra vulnerabilidades de Asignación Masiva).

```
namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class Persona extends Model
{
    use HasFactory;

    // Campos permitidos para inserción masiva
    protected $fillable = [
```

3. El Controlador y la Ruta de la API

A continuación, programaremos la lógica para recibir los datos, validarlos e insertarlos en la base de datos.

El Controlador (PersonaController)

Abre el archivo `app/Http/Controllers/PersonaController.php` y crea el método `store`. Este método manejará las peticiones HTTP POST.

```php
namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Persona;

class PersonaController extends Controller
{
    public function store(Request $request)
    {
        // 1. Validación de los datos entrantes
        $validatedData = $request->validate([
            'nombres' => 'required|string|max:100',
            'apellidos' => 'required|string|max:100',
            'email' => 'required|email|unique:personas,email',
            'telefono' => 'nullable|string|max:20',
        ]);

        // 2. Inserción usando Eloquent
        $persona = Persona::create($validatedData);

        // 3. Respuesta JSON estandarizada (Status 201: Created)
        return response()->json([
            'mensaje' => 'Persona registrada exitosamente',
            'data' => $persona
        ], 201);
    }
}
```

Exponiendo la Ruta de la API

Finalmente, debemos decirle a Laravel qué URL ejecutará nuestro controlador. Abre el archivo `routes/api.php` y registra la ruta:

```php
use App\Http\Controllers\PersonaController;
use Illuminate\Support\Facades\Route;

// El prefijo '/api' se añade automáticamente
Route::post('/personas', [PersonaController::class, 'store']);
```

4. Laboratorio: Consumo desde Postman

Nuestra API ya está programada. Ahora levantaremos el servidor local y usaremos Postman como cliente HTTP para simular una aplicación Frontend insertando datos.

Paso 4.1: Levantar el Servidor

En tu terminal, ejecuta el servidor de desarrollo de Laravel:

```
php artisan serve
```

El servidor estará escuchando en `http://127.0.0.1:8000`.

Paso 4.2: Configuración en Postman

Abre Postman y crea un nuevo Request siguiendo estas especificaciones exactas:

- **Método HTTP:** Selecciona `POST`.
- **URL:** Ingresa `http://127.0.0.1:8000/api/personas`

Paso 4.3: Configuración de Headers (¡Crítico!)

Ve a la pestaña **Headers** en Postman y añade este valor. Esto le dice a Laravel que somos un cliente API y que si hay un error de validación, queremos la respuesta en JSON, no una redirección HTML.

- **Key:** `Accept` | **Value:** `application/json`

Paso 4.4: Envío del Payload (Body)

Ve a la pestaña **Body**, selecciona **raw** y asegúrate de que el formato en el menú desplegable derecho diga **JSON**. Pega el siguiente código:

```
{
    "nombres": "Ana María",
    "apellidos": "Gómez Pérez",
    "email": "ana.gomez@ejemplo.com",
    "telefono": "3001234567"
}
```

Paso 4.5: Ejecución y Verificación

Haz clic en el botón azul **Send**. En la consola de respuesta inferior de Postman deberías ver un **Status 201 Created** y un JSON similar a este confirmando que Eloquent hizo su trabajo:

```
{
    "mensaje": "Persona registrada exitosamente",
    "data": {
        "nombres": "Ana María",
        "apellidos": "Gómez Pérez",
```